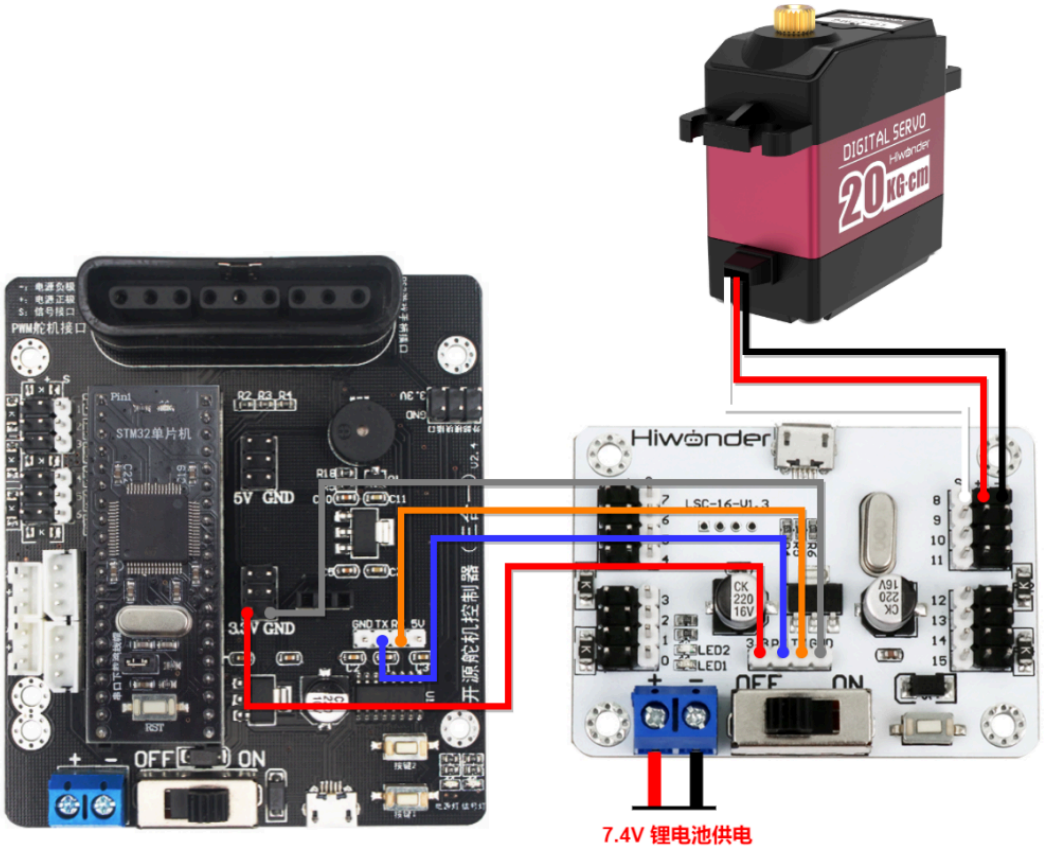
4. 16路舵机控制器二次开发教程-STM32版本开发教程

4.1 准备工作

4.1.1 接线说明

本节示例使用的是16路舵机控制器和开源舵机控制器（控制器上带有STC15W4K32S4单片机），通过7.4V锂电池来供电。将控制器的串口连接至开源舵机控制板的串口上，如下图所示：



> **Note**
>
> **注意：**
>
> 1. **上图舵机接线仅作为接线方法，具体连接接口需参考后文。**
> 2. **如使用我司锂电池，连接锂电池对接线请以红接 + ，黑接-接到 DC 接口。**
> 3. **如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路从而造成短路。**
> 4. **连接串口时，RX与TX要反接。如上图中的两个小图。**

4.1.2 环境配置

在电脑端安装Keil5，软件包位于“**2.软件工具->STM32软件工具**”下。关于Keil5的详细使用，可在对应目录下进行学习。

4.2 案例开发

4.2.1 案例1 控制单个舵机转动

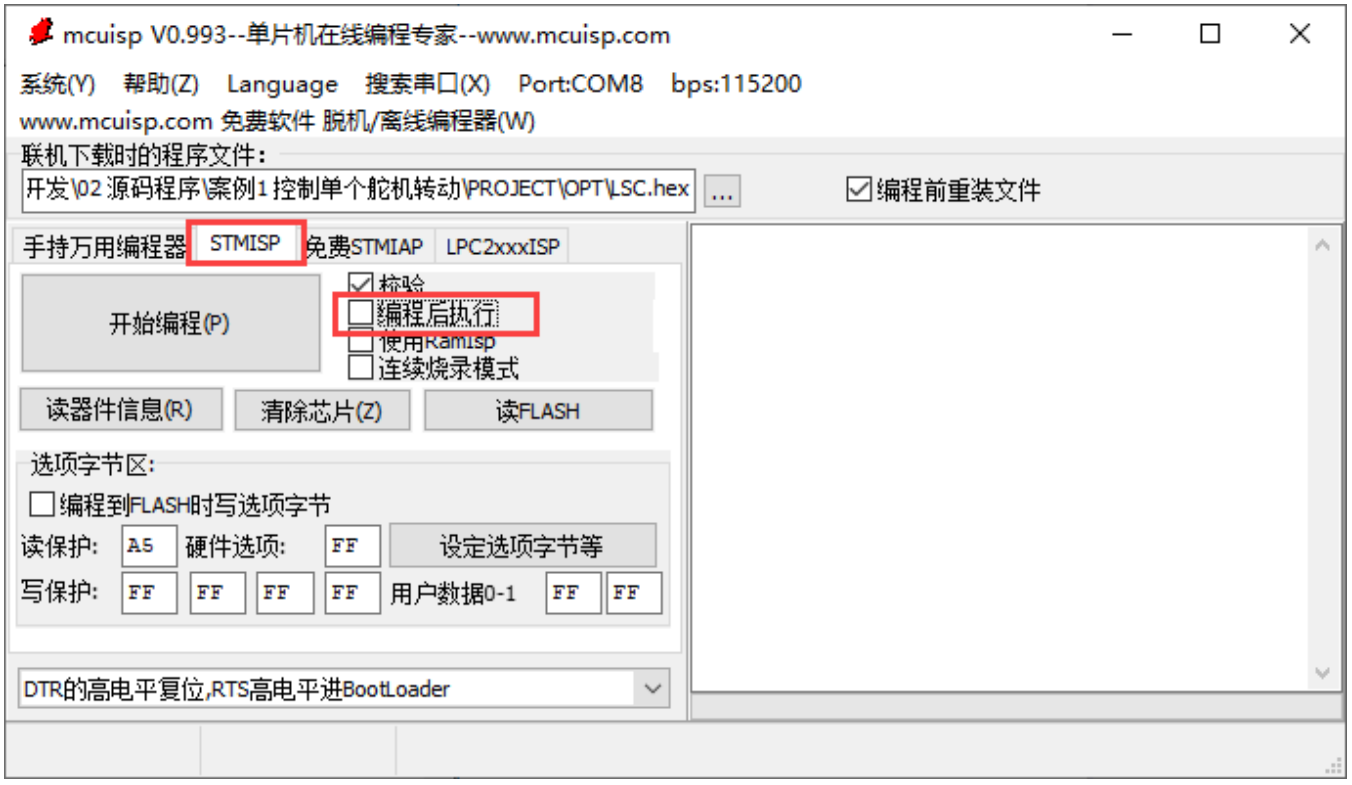
本案例串口向总线控制板发送指令，再通过总线控制板控制舵机转动。

- **运行程序**

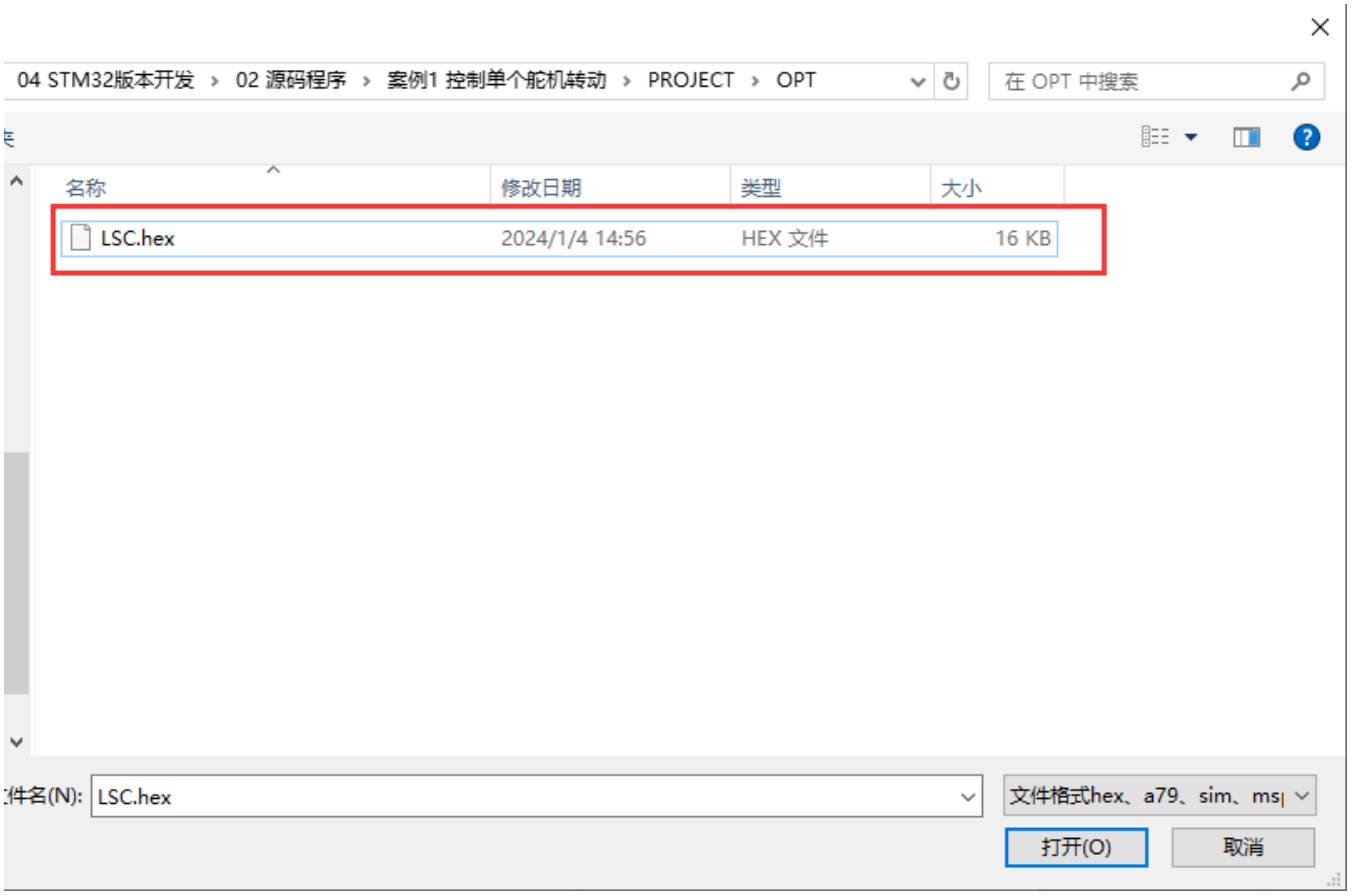
在“**2.软件工具->STM32软件工具**”中打开mcuisp下载工具：



mcuisp配置如下图：

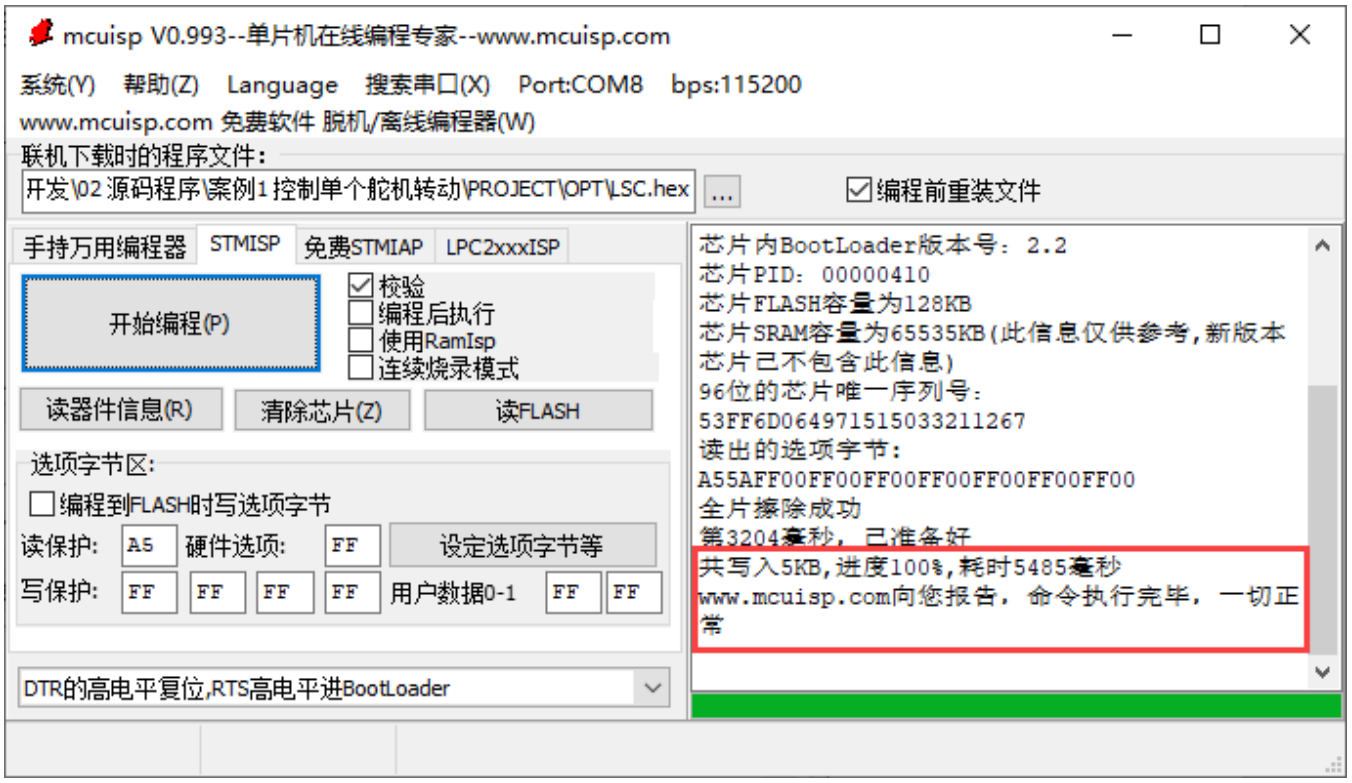


点击“ ... ”，在“**1.教程资料\2.16路舵机控制器二次开发教程\04 STM32版本\02 源码程序\案例1 控制单个舵机转动\PROJECT\OPT**”文件夹下找到hex文件：



打开程序文件后，点击“开始编程”即可将程序下载到STM32单片机上

> **Note**
>
> **注意：在下载程序时，需要先将STM32单片机上的跳线帽拔掉并按下复位按键，当mcuisp右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回STM32单片机。**



下载完成后，打开总线舵机控制器电源开关，按下STM32单片机的复位按键，1号舵机开始转动。

- **实现效果**

程序运行后，连接在控制器上1号接口的舵机会从500的位置到2500的位置之间来回转动，间隔1.2秒。

- **案例程序简要分析**

1. 导入必要功能包

在main.c程序中导入功能包，功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机，如下图所示：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

在main.c文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

2. 串口初始化

在uart.c文件下，初始化串口，配置PA9为TX输出数据，PA10为RX读取数据。配置USART为8为数据位，1位停止位，无校验位，无流控制，可读可写模式。

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA |
RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX   PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX   PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound;//波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
```

```
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

3. 控制舵机转动

开始主程序运行前，调用`SystemInit()`函数初始化系统时钟；调用`delay_init`(72)初始化延时程序；初始化NVIC中断控制器，它的作用是通过系统时钟产生中断来提供延时；初始化串口波特率为9600。

在while循环中，调用`moveServo`函数向总线舵机控制器发送数据，控制舵机转动。

其中，`moveServo`函数接收三个参数，第一个参数代表连接的舵机控制器接口，第二个参数为控制舵机转到指定的脉宽位置，第三个位置为转动至该位置的运行时间。

因此，该例程会控制1号舵机接口的PWM舵机先转动至脉宽为500的位置，再转动至脉宽为2500的位置，以此循环。

```
 int main(void)
 {
    SystemInit();//系统时钟等初始化
    delay_init(72);       //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置NVIC中断分组2:2位抢占优先级，2位响应优先级
    uartInit(9600);//串口初始化为9600

    while(1){
        moveServo(1, 500, 1200); //1号舵机至500位置
        delay_s(2);
        moveServo(1, 2500, 1200); //1号舵机至2500位置
        delay_s(2);
 }
}
```

4.2.2 案例2 控制单个舵机速度

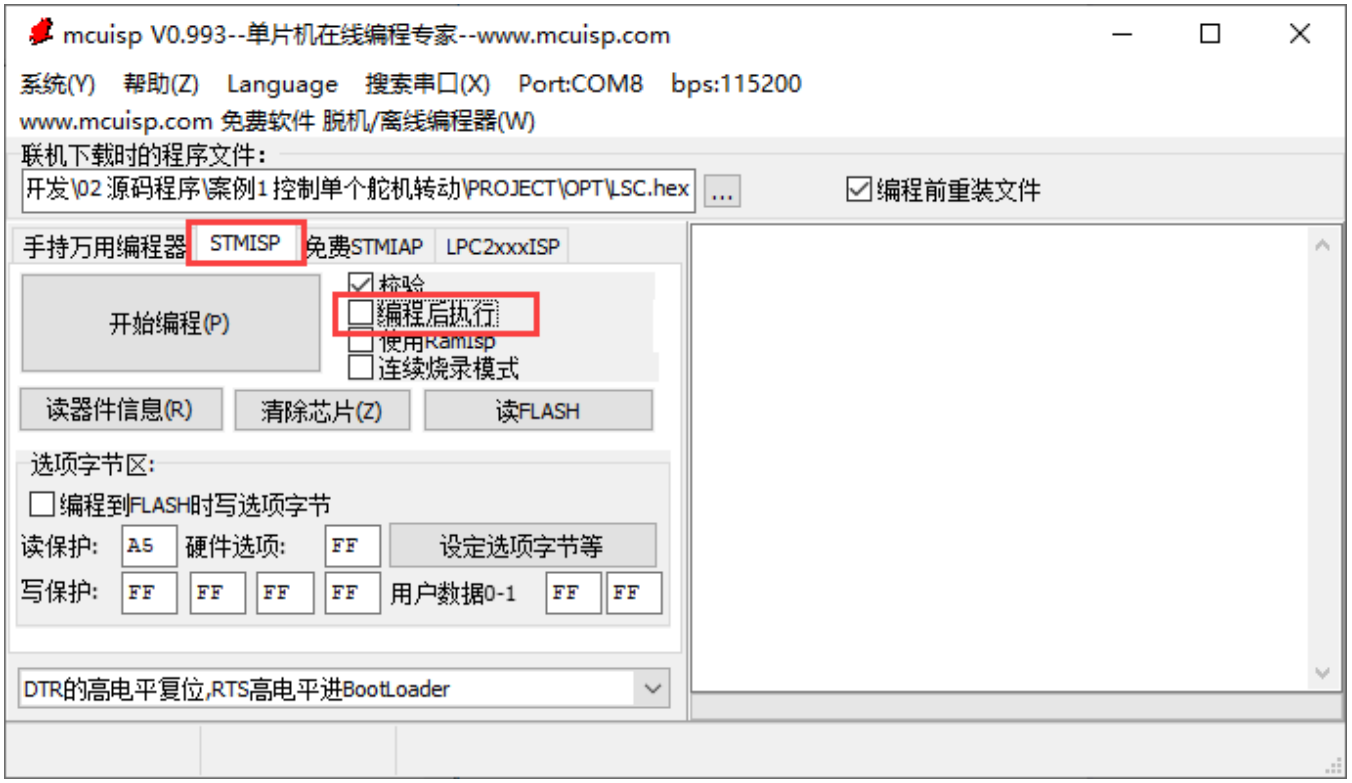
本案例串口向舵机控制器发送指令，再通过控制板控制PWM舵机以不同速度转动。

- **运行程序**

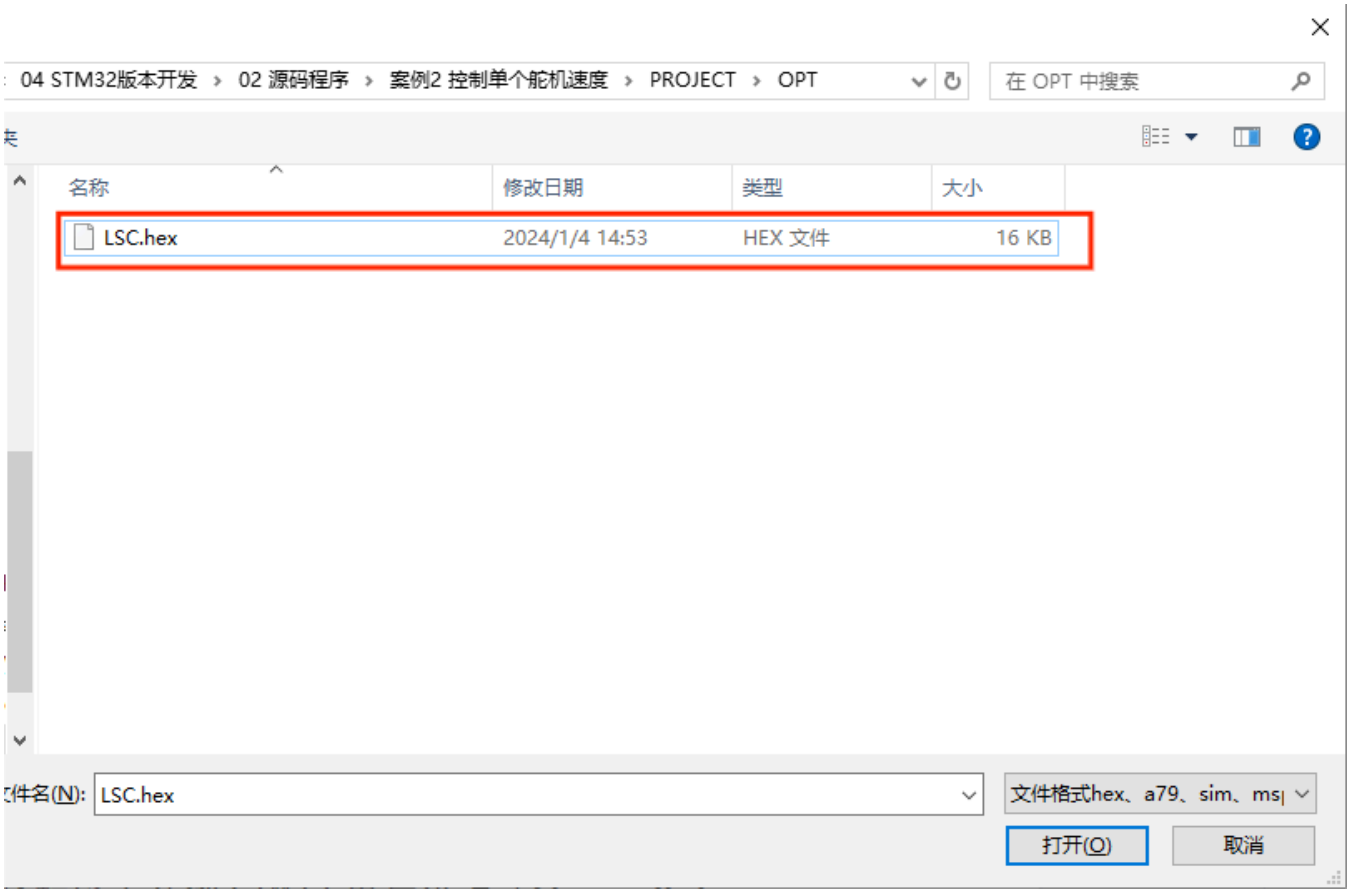
在“**2.软件工具->STM32软件工具**”中打开mcuisp下载工具：



mcuisp配置如下图：

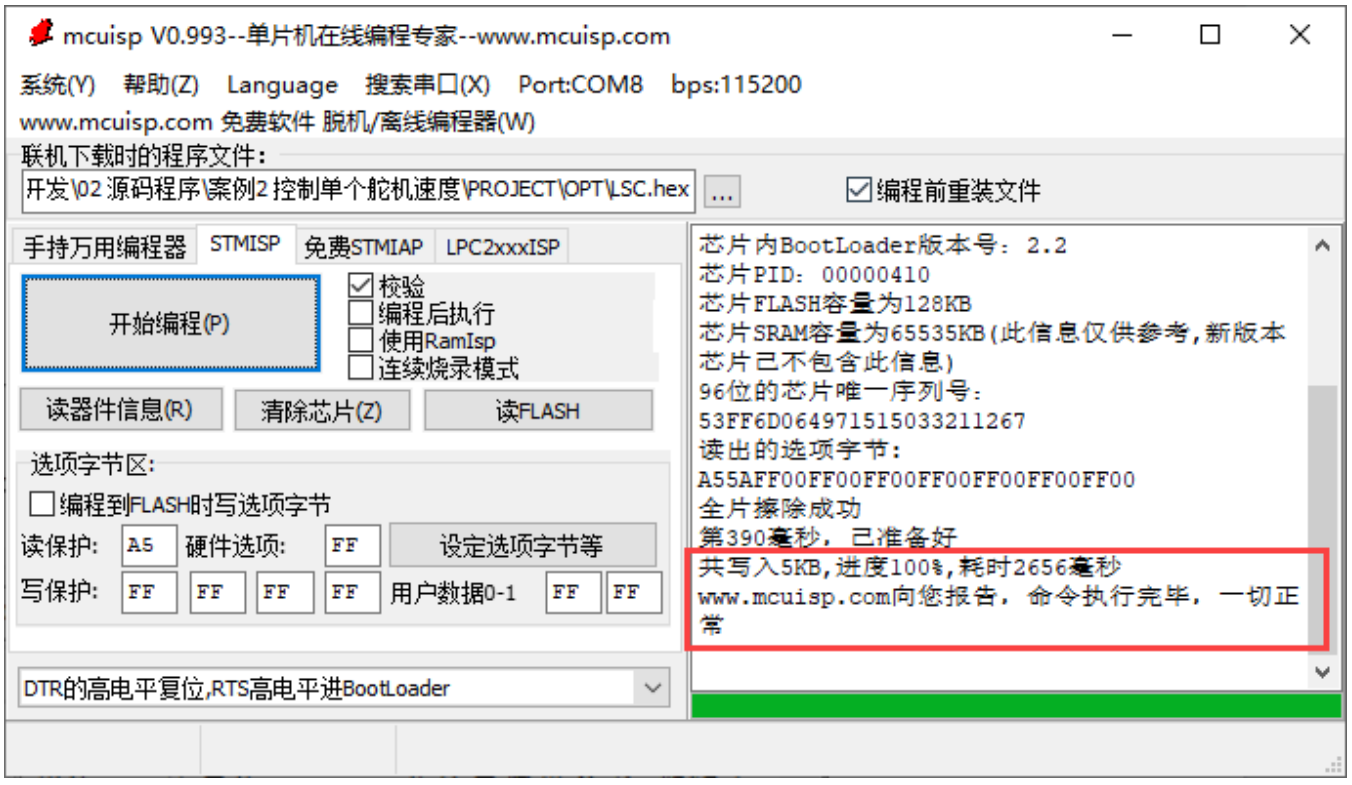


点击“ ... ”，在“**1.教程资料\2.16路舵机控制器二次开发教程\04 STM32版本\02 源码程序\案例2 控制单个舵机速度\PROJECT\OPT**”文件夹下找到hex文件：



打开程序文件后，点击“**开始编程**”即可将程序下载到STM32单片机上

> **Note**
>
> **注意：在下载程序时，需要先将STM32单片机上的跳线帽拔掉并按下复位按键，当mcuisp右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回STM32单片机。**



下载完成后，打开总线舵机控制器电源开关，按下STM32单片机的复位按键，1号舵机开始转动。

- **实现效果**

程序运行后，舵机以1200ms的速度从0位置转动到1000位置，再以500ms速度转动从1000位置转动到0位置。

- **案例程序简要分析**

1. 导入必要功能包

在main.c程序中导入功能包，功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机，如下图所示：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

在main.c文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

2. 串口初始化

在uart.c文件下，初始化串口，配置PA9为TX输出数据，PA10为RX读取数据。配置USART为8为数据位，1位停止位，无校验位，无流控制，可读可写模式。

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA |
RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX   PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX   PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound;//波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
```

```
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

3. 主程序

开始主程序运行前，调用`SystemInit()`函数初始化系统时钟；调用`delay_init`(72)初始化延时程序；初始化NVIC中断控制器，它的作用是通过系统时钟产生中断来提供延时；初始化串口波特率为9600。

在`while`循环中，调用`moveServo`函数向总线舵机控制器发送数据，控制舵机转动。

其中，`moveServo`函数接收三个参数，第一个参数代表连接的舵机控制器接口，第二个参数为控制舵机转到指定的脉宽位置，第三个位置为转动至该位置的运行时间。

因此，程序是先控制舵机以1200ms的速度先转动到500的脉宽位置和2500的脉宽位置，接着再以500ms的位置转动500的脉宽位置和2500的尾款位置，实现控制舵机速度。

```
 int main(void)
 {
    SystemInit();//系统时钟等初始化
    delay_init(72);       //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置NVIC中断分组2:2位抢占优先级，2位响应优先级
    uartInit(9600);//串口初始化为9600


    while(1){
        moveServo(1, 500, 1200); //1号舵机至500位置
        delay_s(2);
        moveServo(1, 2500, 1200); //1号舵机至2500位置
        delay_s(2);
        moveServo(1, 500, 500); //1号舵机至500位置
        delay_s(1);
        moveServo(1, 2500, 500); //1号舵机至2500位置
        delay_s(1);
 }
}
```

4.2.3 案例3 控制多个舵机转动

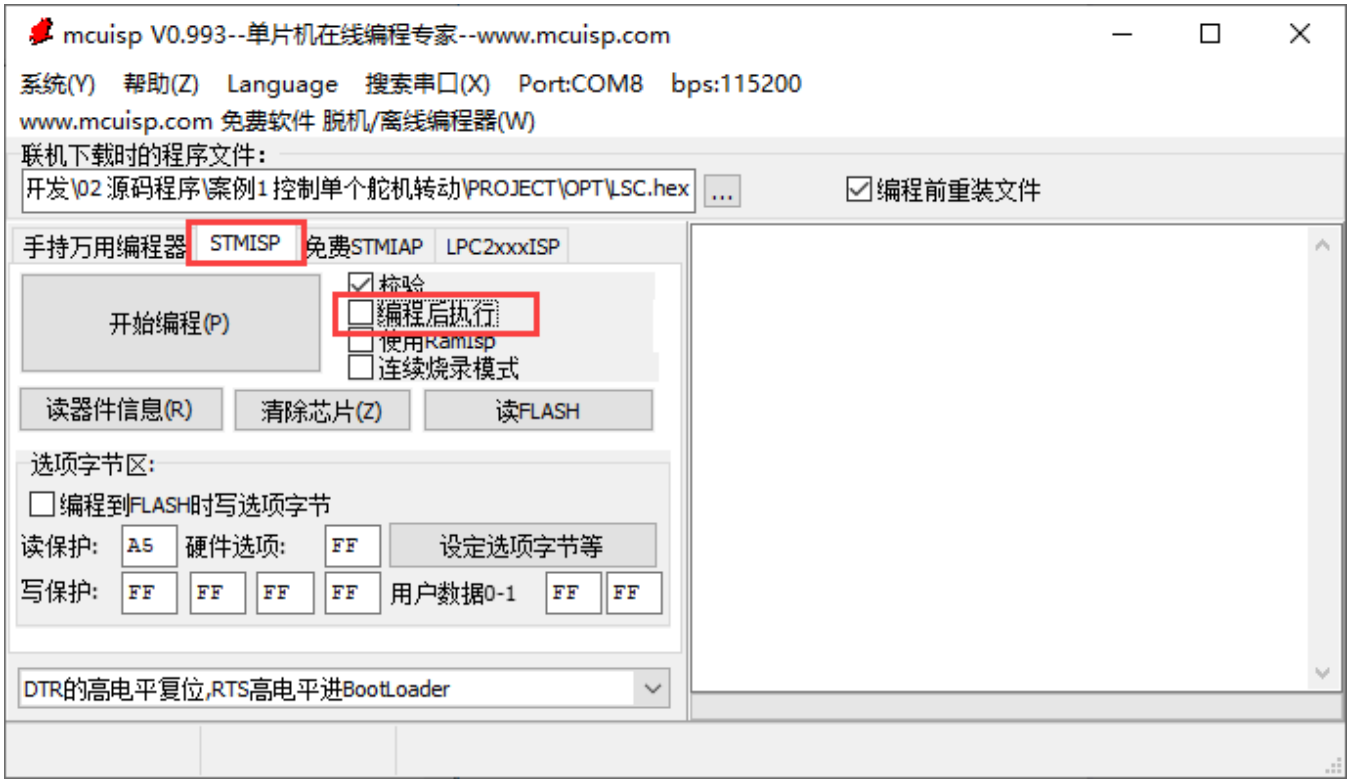
本案例串口向控制板发送指令，再通过控制板控制多个PWM舵机转动。

- **运行程序**

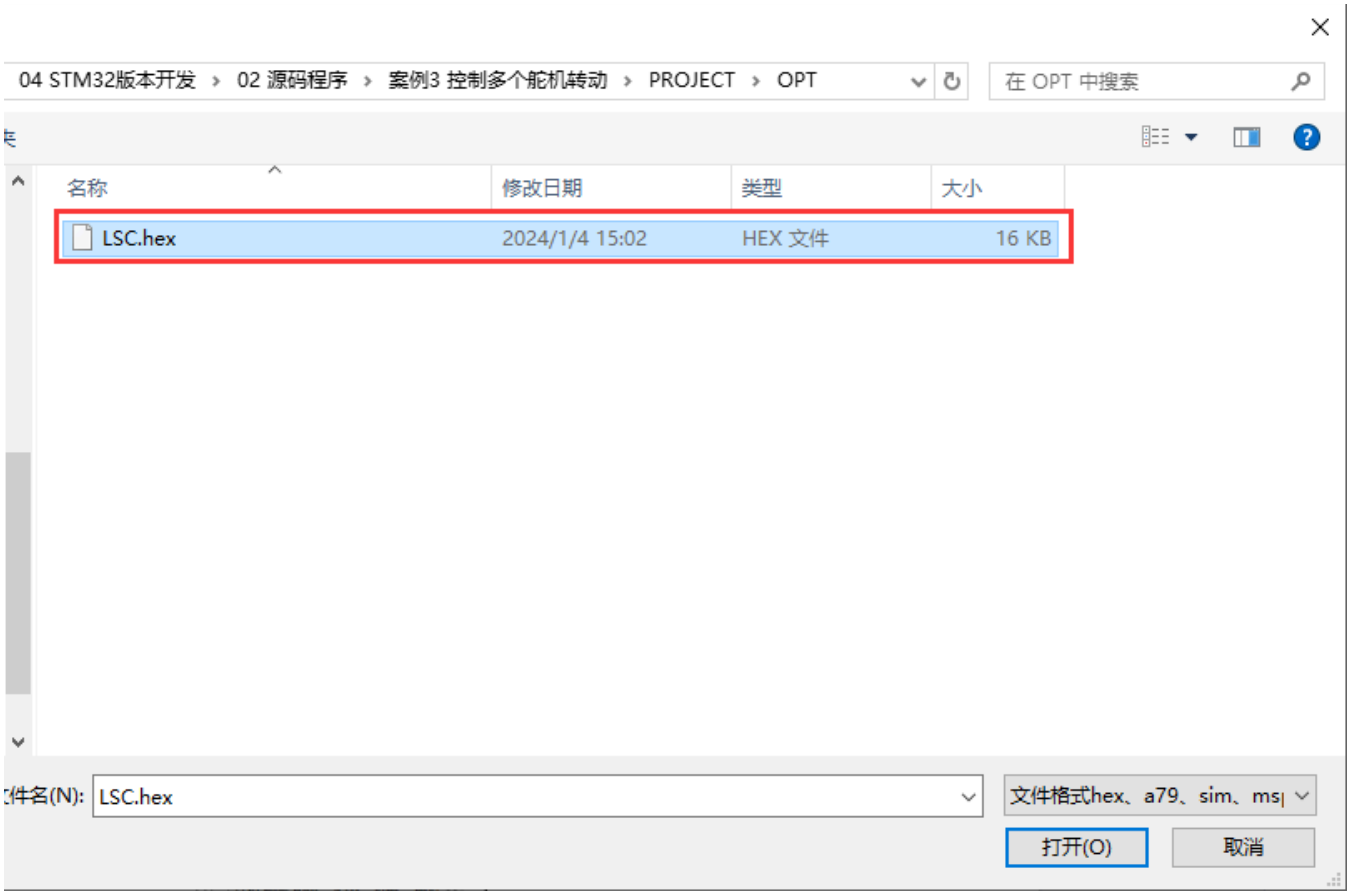
在“**2.软件工具->STM32软件工具**”中打开mcuisp下载工具：



mcuisp配置如下图：

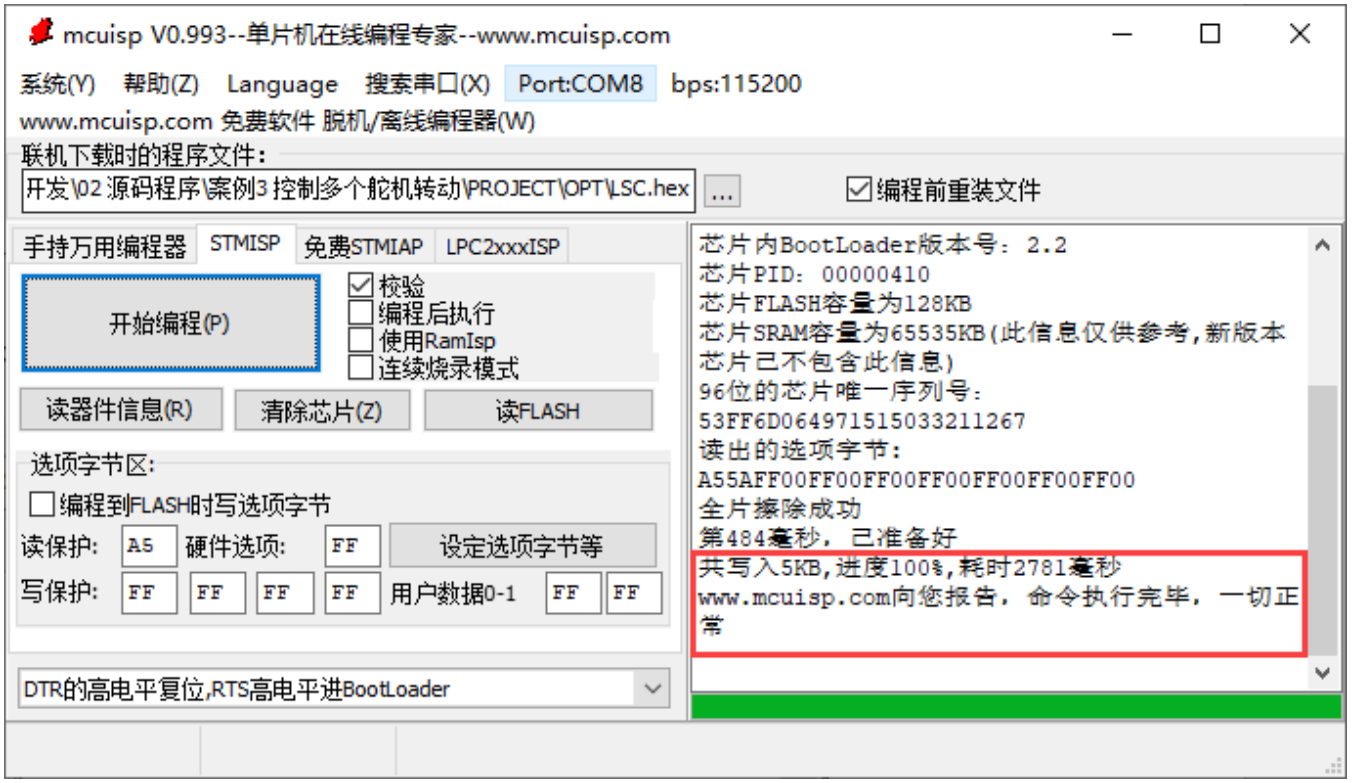


点击“ ... ”，在“**1.教程资料\2.16路舵机控制器二次开发教程\04 STM32版本\02 源码程序\案例3 控制多个舵机转动\PROJECT\OPT**”文件夹下找到hex文件：



打开程序文件后，点击“开始编程”即可将程序下载到STM32单片机上

> **Note**
>
> **注意：在下载程序时，需要先将STM32单片机上的跳线帽拔掉并按下复位按键，当mcuisp右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回STM32单片机。**



下载完成后，打开总线舵机控制器电源开关，按下STM32单片机的复位按键，1号舵机开始转动。

- **实现效果**

程序运行后，舵机1、舵机2和舵机3会从位置500转动到位置2500，以1.5秒为间隔来回转动。（**可操控舵机数量由控制板的接口所定，这里仅操控3个，用户可通过代码尝试控制更多舵机。**）

- **案例程序简要分析**

1. 导入必要功能包

在main.c程序中导入功能包，功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机，如下图所示：

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

在main.c文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

2. 串口初始化

在uart.c文件下，初始化串口，配置PA9为TX输出数据，PA10为RX读取数据。配置USART为8为数据位，1位停止位，无校验位，无流控制，可读可写模式。

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA |
RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX   PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX   PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound;//波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
```

```
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

3. 定义舵机结构体

```
typedef struct _lobot_servo_ {  //舵机ID,舵机目标位置
  uint8_t ID;
  uint16_t Position;
} LobotServo;
```

将需要控制的舵机及信息存放在LobotServo的结构体中，主要用于多个舵机的控制。

3. 控制舵机转动

通过封装在结构体中的舵机信息，通过串口发送给总线控制板，控制板收到指令解析出我要操控的舵机，就会控制相对应的舵机转动到预设的位置。

首先将要控制的舵机ID进行设置，之后通过`moveServosByArray`函数，通过对三个舵机发送指令，控制舵机在500的脉宽位置和2500的脉宽位置来回转动。

```
LobotServo servos[3];

 int main(void)
 {
    SystemInit();//系统时钟等初始化
    delay_init(72);       //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置NVIC中断分组2:2位抢占优先级，2位响应优先级
    uartInit(9600);//串口初始化为9600
    servos[0].ID = 1;            //设置舵机ID
    servos[1].ID = 2;
    servos[2].ID = 3;

    while(1)
    {
        servos[0].Position = 500;        //设置舵机位置为500
        servos[1].Position = 500;
        servos[2].Position = 500;

        moveServosByArray(servos, 3, 1200);      //让3个舵机转动到指定位置，运行时间为800ms
        delay_s(2);

        servos[0].Position = 2500;       //修改位置为2500
        servos[1].Position = 2500;
```

```
        servos[2].Position = 2500;

        moveServosByArray(servos, 3, 1200); //让3个舵机转动到1000的位置，运行时间为800ms
        delay_s(2);

    }
}
```

4.2.4 案例4 舵机中位及偏差调节

本案例串口向舵机控制板发送指令，通过总控制板控制舵机转动中位及偏差位置。

中位为假定即舵机初始的位置，是以此位置为"零点"进行正负角度转动，所以舵机在安装到舵盘前需调至中位再进行安装。

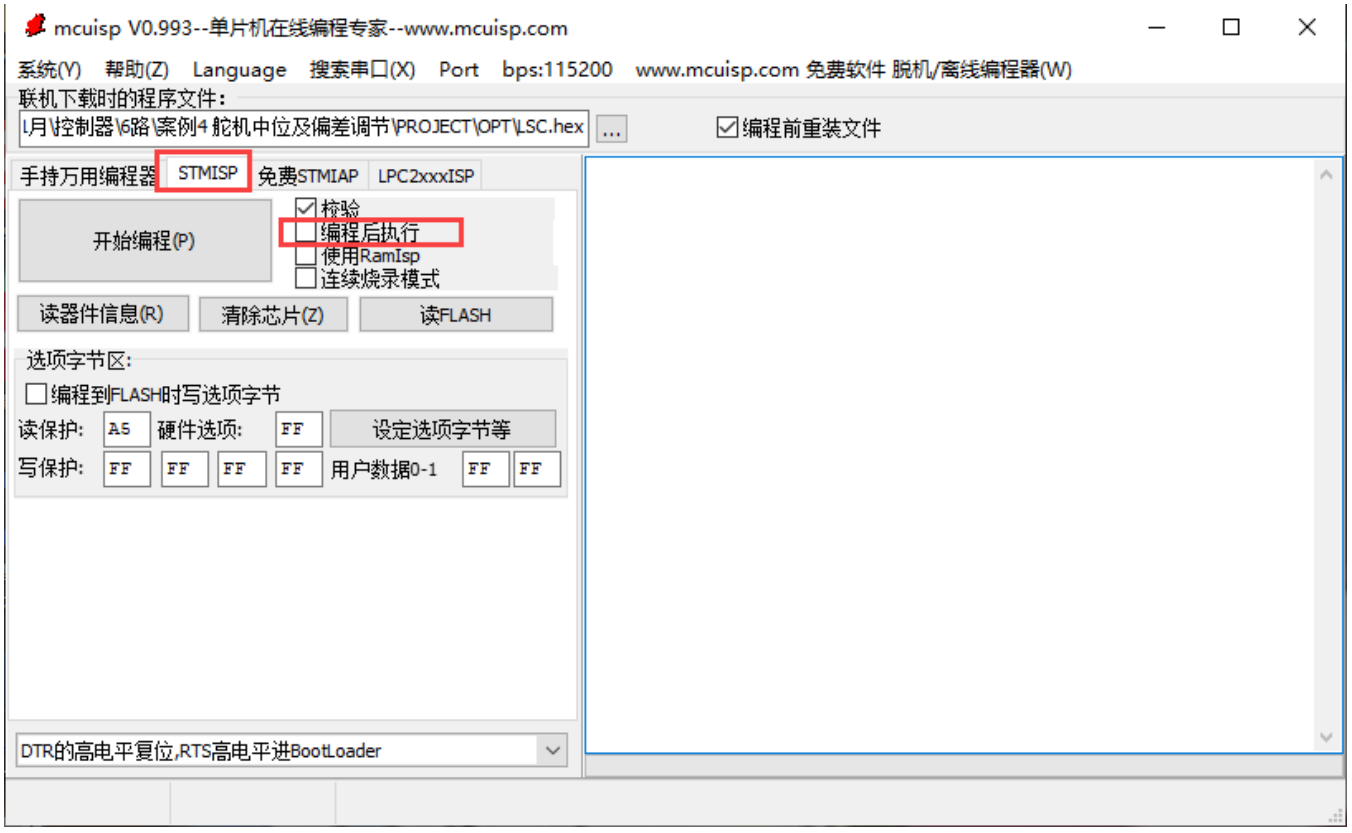
舵机偏差是舵臂上花键的齿的间隔导致，属于机械误差，通过程序可以调节。在机器人或机械臂中，如果不对偏差进行调节，可能会导致机体运动受限，从而影响一些运动效果。

- **运行程序**

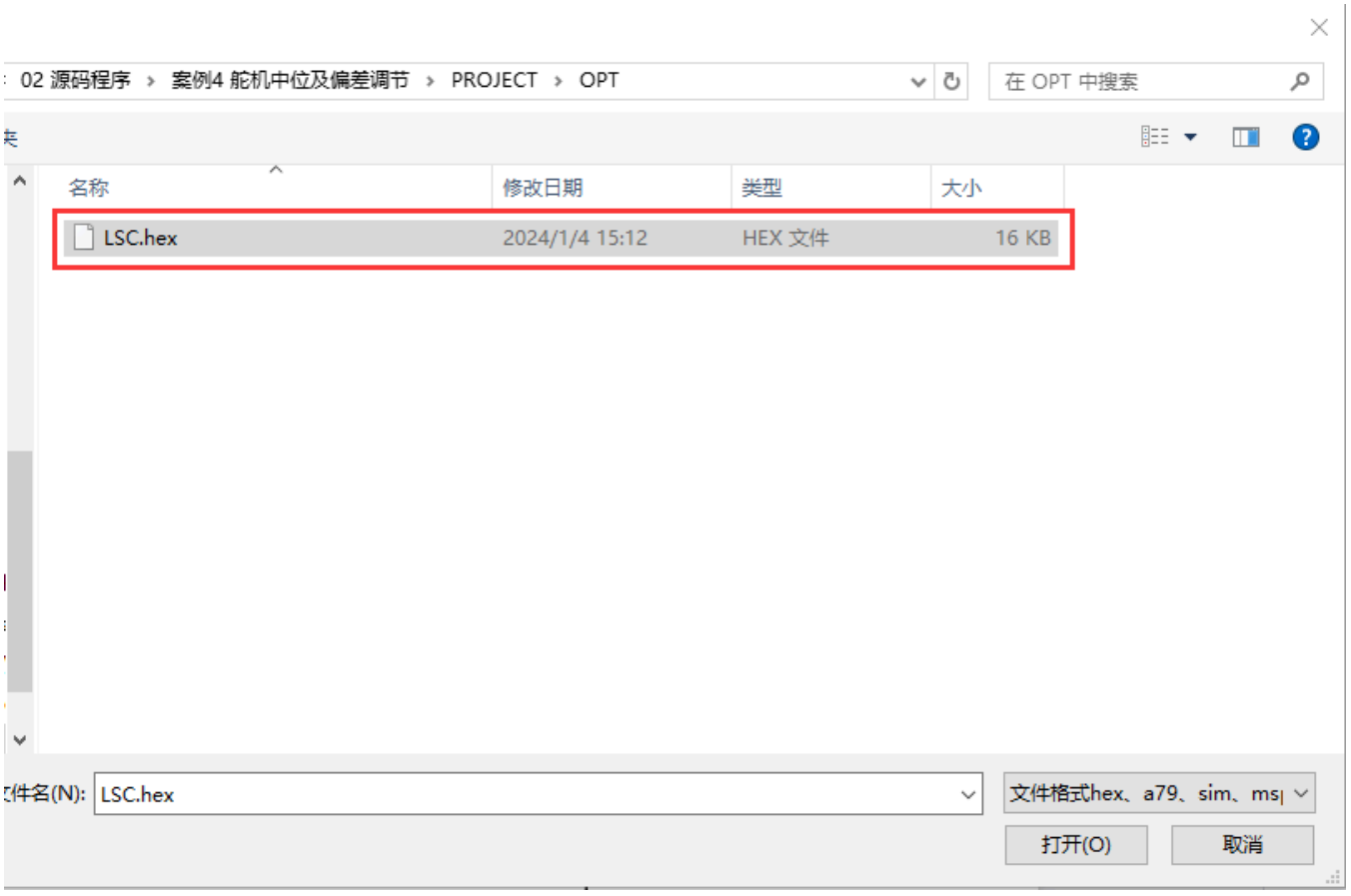
在"**2.软件工具->STM32软件工具**"中打开mcuisp下载工具：



mcuisp配置如下图：



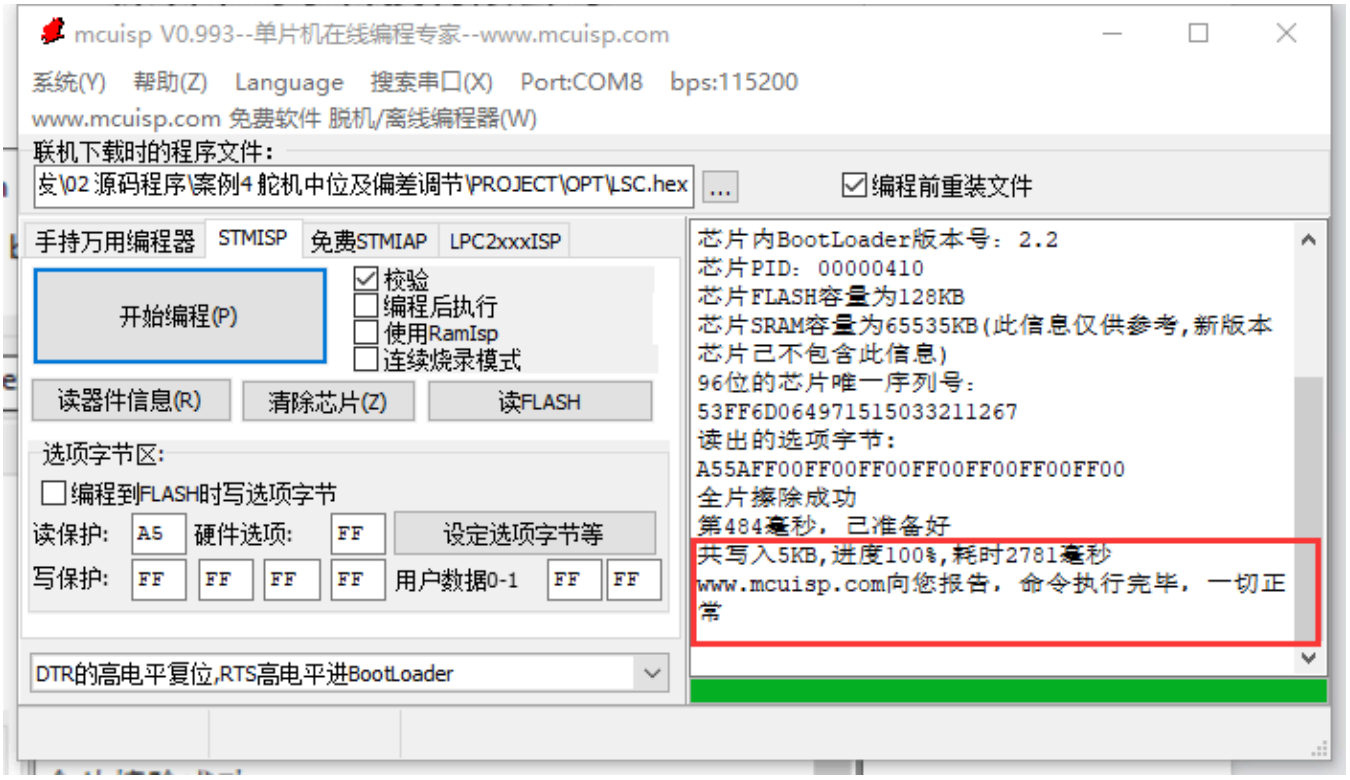
点击“...”，在“**1.教程资料\2.16路舵机控制器二次开发教程\04 STM32版本\02 源码程序\案例4 舵机中位及偏差调节\PROJECT\OPT**”文件夹下找到hex文件：



打开程序文件后，点击“开始编程”即可将程序下载到STM32单片机上

> **Note**

> **注意：在下载程序时，需要先将STM32单片机上的跳线帽拔掉并按下复位按键，当mcuisp右下角显示“命令执行完毕，一切正常”则说明下载完成，最后将跳线帽插回STM32单片机。**



下载完成后，打开总线舵机控制器电源开关，按下STM32单片机的复位按键，1号舵机开始转动。

- **实现效果**

舵机首先回到中位，在短暂的延时过后，转动到偏差角度并保持。

- **案例程序简要分析**

1. 导入必要功能包

在main.c程序中导入功能包，功能包主要封装用于总线控制板通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

```
#include "delay.h"
#include "uart.h"
#include "LobotServoController.h"
#include "bool.h"
```

在main.c文件中，有一个延时函数，该函数用于在单位为：秒（s）的延时。

```
void delay_s(int s)
{
    for(;s>0;s--)
        delay_ms(1000);
}
```

2. 串口初始化

在uart.c文件下，初始化串口，配置PA9为TX输出数据，PA10为RX读取数据。配置USART为8为数据位，1位停止位，无校验位，无流控制，可读可写模式。

```
void uartInit(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA |
RCC_APB2Periph_AFIO, ENABLE);

    //USART1_TX   PA9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART1_RX   PA10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    //USART 配置
    USART_InitStructure.USART_BaudRate = bound;//波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //数据位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //流控
制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //发送接收

    USART_Init(USART1, &USART_InitStructure);
    uartNVICInit();
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    USART_Cmd(USART1, ENABLE);
}
```

3. 控制舵机转动

主程序运行前，定义舵机偏差参数。接着控制1号接口的舵机转动至中位，延时2秒后在中位的基础上再增加一个偏差，让舵机转动到这个偏差的位置，并保持在偏差角度位置。

```
 int main(void)
 {
    int deviation;
    SystemInit();//系统时钟等初始化
    delay_init(72);       //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置NVIC中断分组2:2位抢占优先级，2位响应优
先级
    uartInit(9600);//串口初始化为9600

    moveServo(1, 1500, 500);     //1号舵机，转动到位置1500，用时500ms
```

```
    delay_s(2); //延时程序，这里延时2s
    deviation = 80;
    moveServo(1, 1500+deviation, 200);  //1号舵机，转动到位置1550，用时200ms
    while(1);
}
```